

Proyecto final de Java

Sistema de gestión aeroportuaria — Técnica laboral — Programación orientada a objetos

1. Situación

Una compañía de gestión aeroportuaria administra varios aeropuertos y necesita un programa que le permita consultar la información de sus aeropuertos, las compañías aéreas que operan en cada uno, los vuelos que ofrecen y los pasajeros que han reservado asiento.

Usted debe construir esa aplicación en Java, con **programación orientada a objetos**. El programa funciona por consola y presenta un menú con cinco consultas. Los datos viven en memoria mientras el programa está abierto: no se usa base de datos ni archivos.

Lo que se evalúa de verdad. Este proyecto no se trata de escribir mucho código. Se trata de que las clases estén bien repartidas: que cada clase guarde lo que le corresponde y que la herencia se use donde tiene sentido. Un programa que funcione pero que tenga todo metido en el main obtiene una nota baja.

2. Las clases que debe construir

Clase	Qué guarda (atributos)	Relación con otras clases
Pasajero	nombre, número de pasaporte, nacionalidad	Un pasajero aparece en la lista de un vuelo.
Vuelo	identificador, ciudad de origen, ciudad de destino, precio, número máximo de pasajeros, lista de pasajeros	Un vuelo tiene una lista de Pasajero.
Compañía	nombre, lista de vuelos que ofrece	Una compañía tiene una lista de Vuelo.
Aeropuerto (clase padre)	nombre, ciudad, país, lista de compañías que operan en él	Un aeropuerto tiene una lista de Compañía. Es la clase de la que heredan las dos siguientes.
AeropuertoPúblico (hija)	lo del padre + la cantidad de dinero de la subvención del gobierno	Hereda de Aeropuerto.
AeropuertoPrivado (hija)	lo del padre + la lista con el nombre de las empresas que lo patrocinan	Hereda de Aeropuerto.
Principal (con el main)	la lista de todos los aeropuertos que administra la compañía	Muestra el menú y llama a los métodos de las demás clases.

La estructura en una frase: un **aeropuerto** tiene varias **compañías**, cada compañía tiene varios **vuelos**, y cada vuelo tiene varios **pasajeros**. Los aeropuertos son de dos clases: públicos y privados.

3. Reglas del negocio

- Un aeropuerto es **público o privado, nunca los dos**. Un aeropuerto público tiene subvención del gobierno; uno privado tiene empresas patrocinadoras.
- Un vuelo **no puede aceptar más pasajeros que su número máximo**. Si el vuelo está lleno, el programa debe avisarlo y no reservar.
- El **número real de pasajeros** de un vuelo es cuántos hay en su lista. No lo guarde como un atributo aparte: se calcula con `pasajeros.size()`. Si lo guarda aparte tendrá que acordarse de actualizarlo cada vez, y tarde o temprano se le desincroniza con la lista.
- Una misma compañía puede operar en varios aeropuertos.
- Los atributos deben ser `private`. Se accede a ellos desde afuera únicamente con métodos `get`.

4. El menú de la aplicación

El programa debe mostrar este menú y repetirlo hasta que el usuario escoja salir.

Op.	Nombre	Qué debe hacer
1	Consultar los aeropuertos gestionados	Mostrar por separado los públicos y los privados. De cada uno: nombre, ciudad y país.
2	Ver la financiación de un aeropuerto	El usuario escribe el nombre de un aeropuerto. Si es privado , mostrar las empresas que lo patrocinan. Si es público , mostrar la cuantía de la subvención.
3	Ver las compañías de un aeropuerto	El usuario escribe el nombre de un aeropuerto y el programa lista las compañías que vuelan desde ahí.
4	Ver los vuelos de una compañía en un aeropuerto	El usuario escribe un aeropuerto y una compañía. El programa lista los vuelos de esa compañía con: identificador, ciudad de origen, ciudad de destino y precio.
5	Buscar vuelos entre dos ciudades	El usuario escribe una ciudad de origen y una de destino. El programa muestra todos los vuelos que cubren esa ruta, con su identificador y su precio.
0	Salir	Termina el programa.

La opción 2 es la clave de todo el proyecto. La forma fácil es preguntar de qué tipo es el aeropuerto con un `if` y actuar distinto. La forma correcta es poner en la clase **Aeropuerto** un método abstracto, por ejemplo `informacionDeFinanciacion()`, y que **cada clase hija lo responda a su manera**. Así el menú escribe una sola línea y cada aeropuerto sabe qué contestar. Eso se llama **polimorfismo** y es justamente lo que este proyecto quiere que usted demuestre.

5. Requisitos técnicos obligatorios

- Usar `ArrayList` para todas las listas (compañías, vuelos, pasajeros, patrocinadores). No usar arreglos de tamaño fijo.
- Cada clase va **en su propio archivo .java**, con el mismo nombre de la clase.
- Todos los atributos `private`, con constructor y métodos `get`.
- Usar **herencia**: `AeropuertoPublico` y `AeropuertoPrivado` extienden de `Aeropuerto`, y llaman a `super(...)` en su constructor.
- Usar **al menos un método abstracto** en la clase padre, redefinido con `@Override` en las dos hijas.
- El programa debe arrancar con **datos de prueba ya cargados**: mínimo 3 aeropuertos (al menos uno público y uno privado), 3 compañías, 6 vuelos y 5 pasajeros. Póngalos en un método `cargarDatos()`, no sueltos dentro del `main`.
- El menú se repite hasta que el usuario escoja salir, y no se cae si escribe una opción que no existe.
- Cada opción debe imprimir por consola, como encabezado, **el nombre de los dos integrantes** del equipo.

6. Cómo hacerlo por fases

No empiece por el menú. Empiece por las clases y pruebe cada una antes de seguir a la siguiente.

Fase	Qué se hace	Detalle
Fase 1	Las clases sencillas	Escriba <code>Pasajero</code> y <code>Vuelo</code> . En el <code>main</code> , cree un pasajero y un vuelo, reserve el asiento e imprima los datos. Si eso funciona, siga.
Fase 2	Compañía y aeropuerto	Escriba <code>Compania</code> y la clase padre <code>Aeropuerto</code> . Cree una compañía con dos vuelos y agréguela a un aeropuerto. Imprima la lista.
Fase 3	La herencia	Escriba <code>AeropuertoPublico</code> y <code>AeropuertoPrivado</code> . Cree uno de cada uno y compruebe que <code>informacionDeFinanciacion()</code> responde distinto en cada caso. Esta es la fase que más vale.
Fase 4	Los datos de prueba	Escriba <code>cargarDatos()</code> con los 3 aeropuertos, las compañías, los vuelos y los pasajeros.
Fase 5	El menú	Ahora sí: el <code>do-while</code> con el <code>switch</code> y las cinco opciones. Como las clases ya funcionan, cada opción son unas pocas líneas.
Fase 6	Pruebas y evidencias	Pruebe cada opción con datos que existen y con datos que no existen. Tome las capturas.

7. Cuatro errores que se cometen siempre

Guardar el número de pasajeros como atributo

Si tiene `int numeroPasajeros` aparte de la lista, va a haber un momento en que la lista tenga 3 pasajeros y el contador diga 2. Use `pasajeros.size()`.

Resolver la opción 2 con instanceof o con una bandera

Preguntar `if (aeropuerto instanceof AeropuertoPublico)` funciona, pero es exactamente lo que la herencia viene a evitar. Use un método abstracto.

Vuelos repetidos en la opción 5

Si una compañía opera en dos aeropuertos, al recorrer todos los aeropuertos el mismo vuelo aparece dos veces. Decida qué hacer: mostrar de qué aeropuerto sale, o evitar el duplicado. Lo que no sirve es no darse cuenta.

Comparar textos con ==

Para comparar el nombre que escribió el usuario contra el nombre del aeropuerto hay que usar `equals()` o mejor `equalsIgnoreCase()`. Con `==` casi siempre da falso aunque el texto se vea igual.

8. Qué se entrega

El trabajo se hace **en parejas**. Cada estudiante sube la entrega por separado en la plataforma, indicando el nombre completo de los dos integrantes.

Entregable	Formato	Detalle
1. Código fuente	Carpeta comprimida .zip	Todos los archivos .java. Que compile: si no compila, no se evalúa el funcionamiento.
2. Documento de evidencias	PDF o Word	El código de cada clase y una captura de pantalla de cada una de las cinco opciones del menú en funcionamiento, con la consola visible.
3. Diagrama de clases	Imagen dentro del PDF	Las siete clases con sus atributos, sus métodos y las flechas de herencia y de composición. Hecho a mano y fotografiado también sirve.
4. Sustentación	Oral, en clase	5 minutos por estudiante. Se le pedirá explicar una parte de su propio código y responder por qué lo hizo así.

Sobre la sustentación, para que quede claro desde ahora. Un programa que funcione pero que su autor no pueda explicar no recibe la nota completa, independientemente de dónde haya salido el código. Al revés también vale: un proyecto incompleto pero que usted entienda y sepa explicar saca mejor nota que uno perfecto que no sabe defender. Prepárese para preguntas como «¿por qué esta clase hereda de aquella?» o «¿qué pasa si borro este `super(...)`?»

9. Cómo se califica

Criterio	Puntos	Qué se mira
Diseño de clases y herencia	30	Las siete clases existen, cada una guarda lo que le corresponde, los atributos son privados y las dos clases hijas heredan correctamente con <code>super(...)</code> .
Uso de polimorfismo	15	Hay al menos un método abstracto en <code>Aeropuerto</code> redefinido en las dos hijas, y la opción 2 del menú lo aprovecha en lugar de preguntar por el tipo.
Funcionamiento del menú	25	Las cinco opciones funcionan y muestran exactamente la información que pide el enunciado. El menú se repite y no se cae con una opción inválida.
Evidencias y diagrama	15	Capturas de las cinco opciones, con la consola legible, y diagrama de clases completo.
Sustentación oral	15	El estudiante explica su propio código y responde las preguntas.
TOTAL	100	La nota final se convierte a la escala de 1,0 a 5,0.

Fechas. Entrega del código y las evidencias: _____. Sustentación: _____.

Un consejo. Empiece por las fases 1 a 3. Si llega el día de la entrega y solo tiene hasta la fase 4, entregue eso: vale mucho más que un menú a medio hacer sin clases bien construidas.